

POSTECH

TECH CHALLENGE

Sistema Integrado de Atendimento e Execução de Serviços — Evolução para Microserviços, Kubernetes e CI/CD

Projeto:	Sistema de Oficina Mecânica
Fase:	Fase 2 — Escalabilidade, Resiliência e Automação (SOAT)
Participante:	Lucaz Gabriel Teixeira Denadaí — RM37376
Arquitetura:	Hexagonal + Microserviços (Atendimento e Estoque)
Data de Entrega:	03/07/2026
Backend:	.NET 10 / C#
Banco de Dados:	PostgreSQL 16

Descrição da solução e objetivos da fase

Na Fase 1 foi entregue um MVP monolítico em camadas para gestão de ordens de serviço de uma oficina mecânica. Nesta Fase 2, o objetivo foi evoluir essa aplicação para suportar crescimento de demanda, múltiplas unidades e alta disponibilidade, sem abrir mão de qualidade e organização de código.

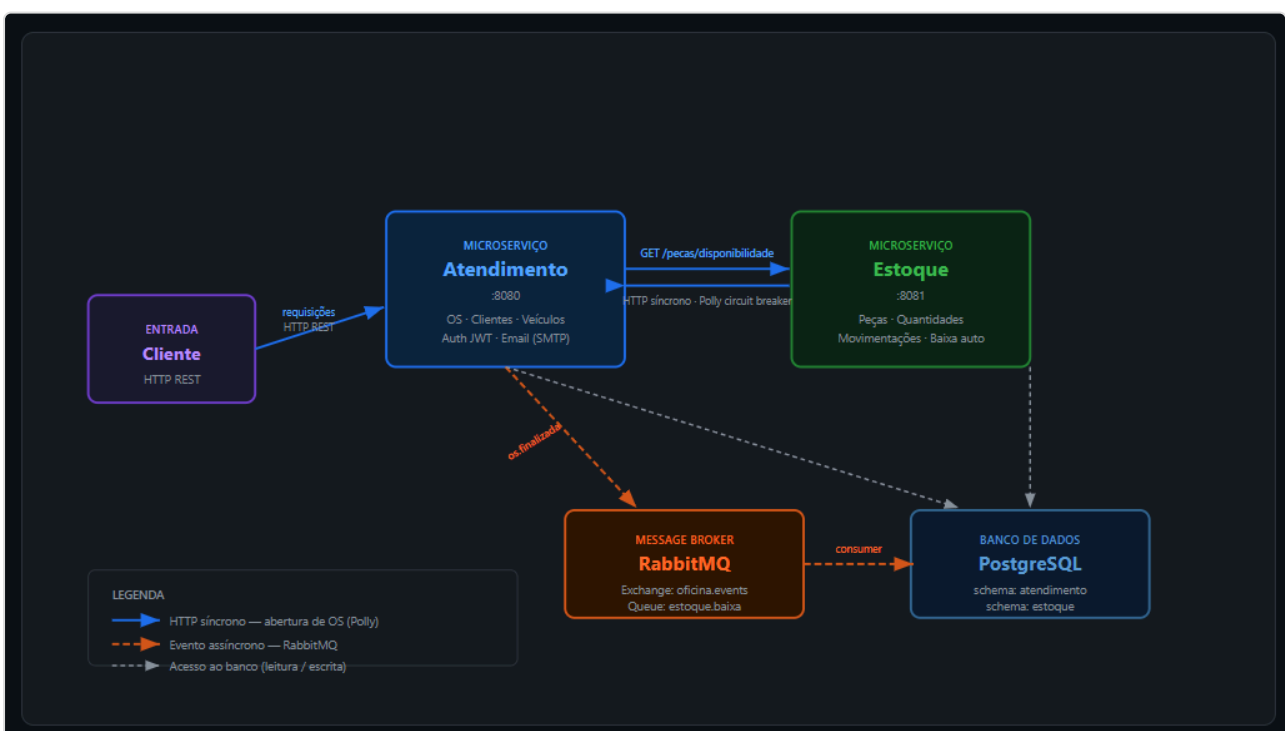
O sistema foi decomposto em dois microsserviços independentes — **Atendimento** e **Estoque** — cada um seguindo Arquitetura Hexagonal (Domain, Application, Infrastructure, API), comunicando-se de forma assíncrona via RabbitMQ para a baixa de peças, com resiliência via retry, dead letter queue e circuit breaker. Toda a infraestrutura passou a ser provisionada como código (Terraform) e orquestrada via Kubernetes, com escalonamento automático (HPA) e um pipeline de CI/CD que builda, testa, publica imagens e faz o deploy de ponta a ponta.

Objetivos alcançados nesta fase

- Refatoração para Arquitetura Hexagonal com separação clara de camadas e inversão de dependência
- Decomposição em dois microsserviços com comunicação síncrona (HTTP) e assíncrona (RabbitMQ)
- APIs de abertura, consulta, aprovação e listagem de OS conforme regras de ordenação e status exigidas
- Containerização via Docker e Docker Compose para desenvolvimento local
- Manifestos Kubernetes completos: Deployments, Services, ConfigMaps, Secrets e HPA
- Infraestrutura como código via Terraform (cluster Kind e PostgreSQL)
- Pipeline de CI/CD (GitHub Actions) cobrindo build, testes, imagem Docker e deploy no cluster
- Observabilidade com Prometheus, Grafana, Loki e Jaeger

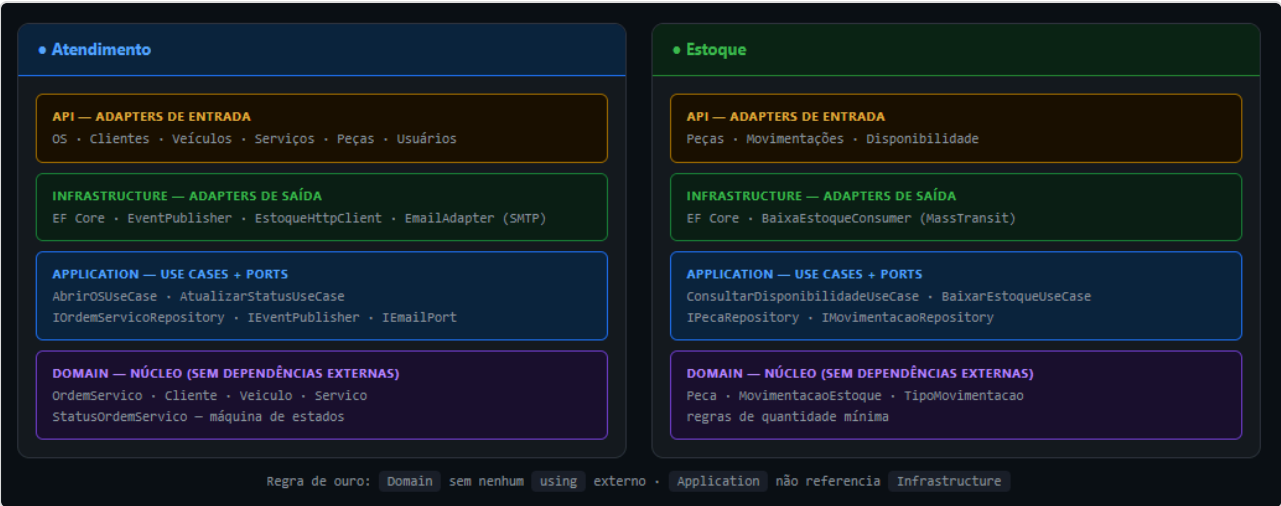
Desenho da arquitetura

Componentes da aplicação e comunicação entre serviços



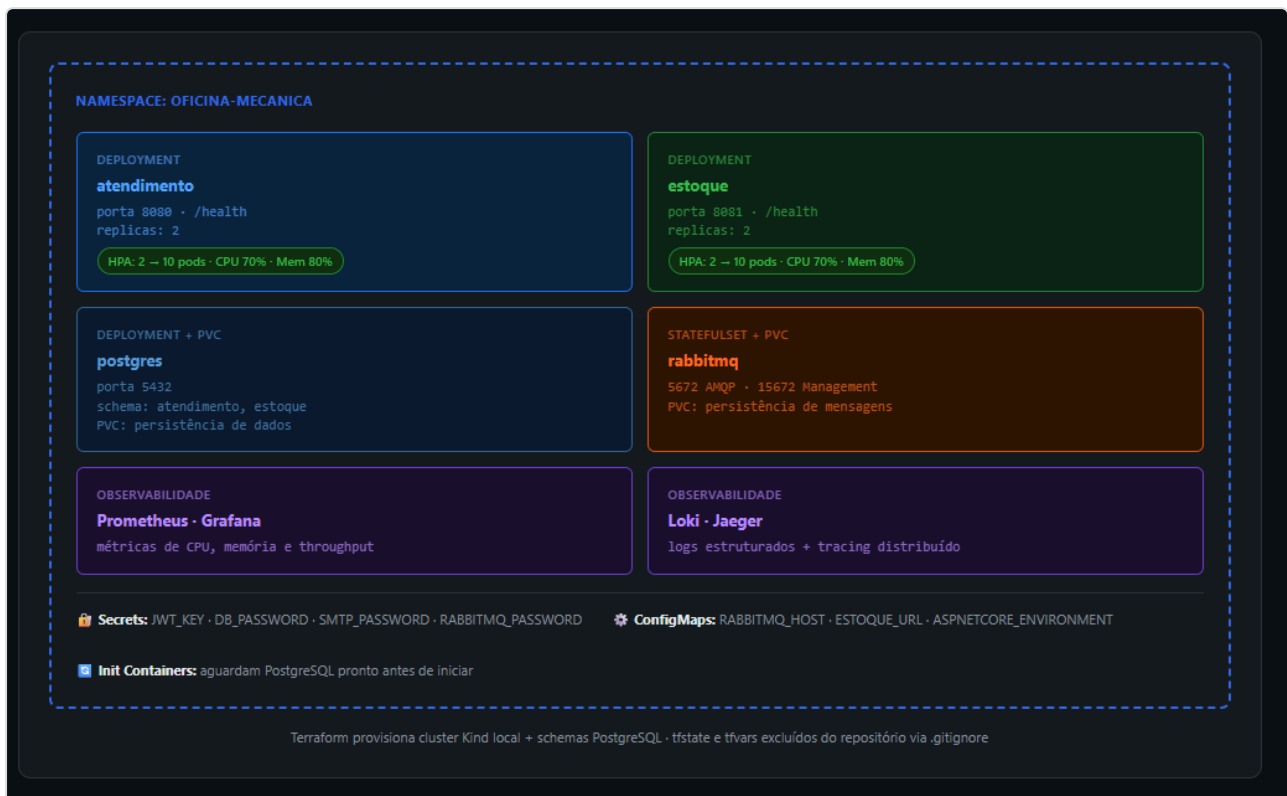
Atendimento expõe a API consumida por atendentes/mecânicos e publica o evento de finalização da OS; Estoque consome o evento via RabbitMQ e executa a baixa das peças.

Arquitetura Hexagonal por serviço



Cada microserviço segue quatro camadas — Domain, Application, Infrastructure e API — com a dependência sempre apontando para dentro.

Infraestrutura provisionada (Kubernetes)



Pods de Atendimento e Estoque no namespace oficina-mecanica, com HPA (2–10 réplicas), PostgreSQL e RabbitMQ com PersistentVolumeClaims, e stack de observabilidade no namespace observabilidade.

Fluxo de deploy — CI/CD



build-and-test → infra (Terraform provisiona cluster e banco) → docker (build e push das imagens no GHCR) → deploy (kubectl apply dos manifestos), com jobs encadeados via needs.

Principais decisões arquiteturais

Ao longo da evolução do projeto, alguns problemas concretos exigiram decisões conscientes de arquitetura. Um resumo do raciocínio de cada uma:

Por que dois microsserviços em vez de um monolito modular

Em horários de pico, o volume de novas ordens de serviço cresce muito mais que o de movimentações de estoque — num monolito, escalar uma parte significa escalar tudo junto. Além disso, uma falha temporária no processamento de estoque não pode travar a finalização de uma OS. A solução foi separar Atendimento e Estoque em serviços independentes, comunicando-se de forma assíncrona (RabbitMQ) para a baixa de peças, com retry automático e fila de erro — e comunicação síncrona apenas na checagem de disponibilidade ao abrir a OS.

Como saber se uma baixa de estoque falhou

Depois de esgotados os retries automáticos, não havia forma de responder "essa OS teve a baixa de estoque processada?" — o RabbitMQ não vincula a falha à OS nem mantém histórico consultável. A decisão foi gravar as falhas definitivas em uma tabela própria no banco do Estoque, expondo um endpoint de consulta por OS, sem sobrecarregar o banco (só grava depois que todas as tentativas falham).

Quem é o dono do cadastro de peças

Atendimento e Estoque cadastravam peças de forma independente, cada um gerando seus próprios IDs — isso exigia sincronização manual e causava falhas silenciosas na baixa automática quando os IDs não coincidiam. A decisão foi tornar o Estoque a única fonte da verdade: o Atendimento consulta o Estoque via API para obter dados da peça, eliminando a duplicidade de cadastro.

Onde rodar o cluster Kubernetes

O ambiente precisa ser recriado do zero por qualquer avaliador, de forma rápida e sem depender de conta em nuvem ou conexão de internet. Optamos por provisionar o cluster localmente (Kind) via Terraform, mantendo a estrutura em módulos para que, em um cenário real, o mesmo código pudesse ser adaptado para um provedor cloud sem reescrever a aplicação.

Como o pipeline consegue fazer deploy no cluster local

O runner padrão do GitHub Actions roda na nuvem da GitHub e não enxerga o cluster local — qualquer tentativa de deploy falha por falta de acesso de rede. A solução foi usar um runner self-hosted, instalado na mesma máquina do cluster, apenas para o job de deploy; build e testes continuam rodando no runner padrão para manter um ambiente limpo e reproduzível.

Um banco ou dois para os microsserviços

A prática recomendada em microsserviços é cada serviço ter seu próprio banco isolado, mas isso dobraria a complexidade operacional (dois Deployments, dois PVCs, duas migrations) para o volume real do projeto. Optamos por uma única instância PostgreSQL com schemas separados por serviço, aceitando conscientemente que o banco não escala de forma independente — a escala de aplicação já é resolvida pelo HPA — com um caminho de migração documentado caso o volume cresça no futuro.

Instruções de execução

Execução local

```
git clone https://github.com/LucazDenadai/tech-challenge.git
cd tech-challenge && cp .env.example .env
docker compose up --build
```

Swagger: <http://localhost:8080/swagger> (Atendimento) e <http://localhost:8081/swagger> (Estoque).

Deploy em Kubernetes

```
kubectl apply -f k8s/namespace.yaml
Criar secrets de conexão e JWT · kubectl apply -f k8s/ -n oficina-mecanica
Verificar rollout e HPA com kubectl rollout status e kubectl get hpa -n oficina-mecanica.
```

Provisionamento com Terraform

```
cd infra && terraform init
terraform apply -var="db_password=suasenha"
```

Instruções completas e detalhadas de cada etapa estão no README.md do repositório.

Links da entrega

Repositório GitHub

<https://github.com/LucazDenadai/tech-challenge>

Collection completa das APIs (Postman)

https://github.com/LucazDenadai/tech-challenge/blob/main/docs/oficina-mecanica.postman_collection.json

Documentação Swagger (com o ambiente em execução)

<http://localhost:8080/swagger> · <http://localhost:8081/swagger>

Vídeo demonstrativo (YouTube)

<https://youtu.be/Gr7zDqgdNhs>

Demonstra deploy da aplicação, execução do CI/CD, consumo das APIs e escalabilidade automática (HPA) sob carga.